

layer) needs more effectively. This approach is akin to hyperparameter tuning, but instead of training multiple models, we let the current model learn an optimal function map.

Parameterized Activation Functions. To overcome the limitations of fixed activation functions, researchers have focused on parameterized activation functions. It introduces learnable parameters that can be adapted during training, which enables the network to optimize the activation function for specific tasks. This adaptability allows networks to better capture data characteristics, improving convergence rates, expressiveness, and task-specific performance (Goyal et al., 2020). For example, Parametric ReLU (PReLU) extends the Leaky ReLU’s flexibility by making the slope parameter p trainable. It is defined as:

$$\text{PReLU}(x) = \begin{cases} x & \text{if } x > 0, \\ px & \text{if } x \leq 0, \end{cases}$$

in the output range of $(-\infty, \infty)$. Like Leaky ReLU, this approach mitigates the dead neuron problem, allowing the network to adaptively determine the gradient flow for negative inputs and improve convergence (Dubey et al., 2022). In general, by adjusting for parametrization, PReLU reduces sensitivity to initialization and increases performance across tasks. Similarly, SPLASH (Simple Piecewise Linear and Adaptive with Symmetric Hinges) (Tavakoli et al., 2020) uses a piecewise linear activation to enhance flexibility. By employing symmetry, grounding, and fixed hinge points, SPLASH simplifies the learning process by reducing the parameter space from $3S + 2$ to $S + 1$, enabling faster optimization and improved generalization. The activation of a hidden unit $h(x)$ in SPLASH is formulated as $S + 1$ max functions with S symmetric offsets, where S is an odd number, and one of the offsets is zero:

$$h(x) = \sum_{s=1}^{(S+1)/2} a_+^s \max(0, x - b^s) + \sum_{s=1}^{(S+1)/2} a_-^s \max(0, -x - b^s),$$

where the learned parameters a_+^s, a_-^s determine the slope of each line segment, and the hinge locations $b^s, -b^s$ are fixed. Despite having fewer parameters, though, SPLASH remains capable of approximating a wide range of non-linear functions, and has demonstrated effectiveness in tasks requiring robust generalization and resistance to adversarial attacks. For instance, in classification problems, SPLASH has outperformed traditional activations like ReLU and Leaky ReLU in terms of both accuracy and adversarial robustness (Tavakoli et al., 2020). The symmetry and continuity of the activation function result in smoother decision boundaries, enhancing the model’s resilience to adversarial perturbations. Finally, SWISH (Ramachandran et al., 2017) blends linear and sigmoid components, providing smoother transitions between activations. The SWISH activation function is defined as:

$$\text{SWISH}(x; \beta) = x \cdot \sigma(\beta x),$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function, and β is a constant or trainable parameter that controls the degree of non-linearity. When $\beta \rightarrow 0$, SWISH behaves like a linear activation function, and as $\beta \rightarrow \infty$, it becomes similar to the ReLU activation (Ramachandran et al., 2017). As such, SWISH can be loosely viewed as a smooth function that nonlinearly interpolates between the linear and the ReLU functions. One of the key advantages that differentiates SWISH from other activation functions is its smoothness and non-monotonicity. The differentiability of SWISH ensures that gradient-based optimization algorithms can update weights effectively without encountering abrupt changes, as often seen in ReLU due to its non-smooth nature at $x = 0$. The non-monotonicity introduced by the sigmoid term also allows SWISH to capture more complex relationships in the data compared to monotonic functions like ReLU (Nair & Hinton, 2010). Finally, the inclusion of the trainable parameter β allows the network to dynamically adjust the activation function during training. This flexibility ensures SWISH can adapt to the characteristics of the data and the architecture of the network, potentially resulting in better optimization and generalization. In experiments, SWISH demonstrated superior performance across various deep learning tasks, particularly in ImageNet with architectures such as MobileNet and Inception-ResNet (Szegedy et al., 2016).

General Parametric Activation Function. Hu et al. (2021) takes the most general approach to researching parametrized activation functions. They study a model which parametrizes any "traditional"

activation function $\sigma(\cdot)$, defining the layer-specific function as:

$$\sigma_i(a_i, b_i, c_i, d_i, z) = b_i \sigma(a_i z + c_i) + d_i,$$

where $z = wx + b$ denotes the weighted sum of inputs, including the bias term. These approaches significantly improve performance by tailoring activation behavior to specific tasks, at a relatively cheap cost of only a few parameters per layer. However, parameterized activations assume that a single functional form, albeit adjustable, can optimally serve all layers in a network. This assumption limits their ability to capture the diverse requirements of different layers. Layers closer to the input may require smoother transformations for feature extraction, while deeper layers may benefit from sharper non-linearities for decision boundaries. Our work differs by addressing these layer-specific needs directly, providing greater flexibility without the complexity of designing or training parameterized families.

Discrete Function Selection and Activation Routing. The most similar approach to our work would be that of [Manessi & Rozza \(2018\)](#), who consider linear combinations of multiple activations, enabling smooth transitions and simplifying optimization. Our approach could actually be seen as simply a discrete restriction of their method. However, while effective at addressing layer-specific needs, their method does not provide the interpretability of ours, as the resulting activation is a weighted blend rather than a distinct choice. Furthermore, the blend was restricted to the functions: Identity, ReLU, and Tanh with the ReLU function often receiving a larger score. We experience this bias ourselves, and discovered it is due to the magnitude differences of the activations, where ReLU’s unbounded nature causes it to be preferred in optimization procedures, regardless of the data structure. However, unlike [Manessi & Rozza \(2018\)](#), we were able to discover and address this bias due to the interpretability of our method and the structure of our experiments. After our correction, our model successfully picks Sigmoid or Tanh if they are more suitable representations, while such validation is not available in prior literature.

In summary, prior works have explored static, parameterized, or blended activation mechanisms. Our work departs from these by introducing a modular, interpretable, and efficient framework for activation function selection via discrete routing.

3 Background

Neural Network Preliminaries. We consider feedforward neural networks as compositional mappings $F : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_y}$, defined via a sequence of layers:

$$F(x) = g_N \circ g_{N-1} \circ \cdots \circ g_1(x),$$

where each layer $g_j : \mathbb{R}^{d_{j-1}} \rightarrow \mathbb{R}^{d_j}$ consists of an affine transformation followed by a component-wise non-linearity:

$$g_j(z) = \sigma_j(W_j z + b_j), \quad W_j \in \mathbb{R}^{d_j \times d_{j-1}}, \quad b_j \in \mathbb{R}^{d_j}.$$

Here, the activation function $\sigma_j : \mathbb{R} \rightarrow \mathbb{R}$ acts coordinate-wise on the vector, i.e., $(\sigma_j(w))_k = \sigma_j(w_k)$. While classical networks use a fixed activation $\sigma_j \equiv \sigma$ across all layers, our work focuses on the flexible and learnable selection of σ_j from a finite set, discussed further in Section 3.2.

The network is trained on labeled data $\{(x_i, y_i)\}_{i=1}^m$, minimizing a loss function $\ell : \mathbb{R}^{d_y} \times \mathbb{R}^{d_y} \rightarrow \mathbb{R}_+$. The empirical risk objective is:

$$\mathcal{L}(\{W_j, b_j\}) = \sum_{i=1}^m \ell(F(x_i), y_i),$$

optimized via gradient-based methods. The dominant computational workhorse here is backpropagation, which leverages the chain rule to compute gradients efficiently ([Rumelhart & McClelland, 1987](#)). This backpropagation process depends on the form and differentiability of each σ_j .

Activation Function Design. Activation functions introduce the non-linearity required for deep networks to approximate highly complex mappings as compiled in [Dubey et al. \(2022\)](#). While historically fixed across

all layers, growing model depth and diversity of layer function suggest that more sophisticated, possibly adaptive, designs can offer improved expressiveness and trainability.

Our framework allows each layer to select an activation from a set $\{\sigma^{(1)}, \dots, \sigma^{(p)}\}$. In this paper, we consider the following activation functions:

3.0.1 ReLU

One of the most popular choices of activation functions is the Rectified Linear Unit (ReLU). It is defined as

$$\text{ReLU}(x) = \max(0, x).$$

ReLU is computationally efficient and mitigates the vanishing gradient problem for positive inputs. However, the main limitation with ReLU is that all the negative values become zero, and some gradients can be fragile during training, resulting in dead neurons (Agarap, 2019).

3.0.2 Sigmoid

Sigmoid is defined as

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

It maps inputs to $[0, 1]$, making it suitable for probabilistic outputs. Despite its interpretability, it faces the “vanishing gradient problem,” and the saturating of gradients for large input values poses challenges during training (Boullé et al., 2020).

3.0.3 Tanh

The Tanh activation function is defined as

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Tanh is a scaled and shifted version of the sigmoid function that maps inputs to $[-1, 1]$, providing zero-centered outputs, which often help with optimization. However, it also suffers from the vanishing gradient problem for large positive or negative inputs, where the function saturates and the derivative approaches zero. It also requires exponentials for computation, which could be slower in training and inference (Dubey et al., 2022).

3.0.4 Leaky ReLU

Leaky ReLU is one of the variations of the ReLU function that addresses the dead neurons problem. It is defined as

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0, \\ kx & \text{if } x \leq 0, \end{cases}$$

where k is a small positive constant. Unlike ReLU, Leaky ReLU has a small gradient k for negative inputs, therefore preventing the dead neuron problem. It also mitigates the gradient saturation problem for positive inputs. However, Leaky ReLU requires tuning another hyperparameter, and it does not always outperform the original ReLU activation function (Boullé et al., 2020; Tavakoli et al., 2020).

3.0.5 Identity

The identity function is defined as

$$f(x) = x.$$

The identity function retains linearity, serving as a baseline for testing the network’s behavior with non-transformative activations (Dubey et al., 2022). The Identity function serves as an important sanity check in many of our experiments.